

IF25-22017

PENGEMBANGAN APLIKASI MOBILE

PERTEMUAN 4

State Management dan MVVM

Reactive UI dengan State, ViewModel, dan Architecture Pattern

Program Studi Teknik Informatika

Institut Teknologi Sumatera

Tahun Akademik 2024/2025

[Logo
Kemendikbud]

[Logo Diktisaintek]

[Logo ITERA]

OUTLINE PERTEMUAN

01

Review Minggu Lalu

Compose Basics, Layouts, Modifiers

02

State dalam Compose

remember, mutableStateOf, State

03

Recomposition

Bagaimana UI di-update

04

State Hoisting

Mengangkat state ke parent

05

MVVM Pattern

Model-View-ViewModel Architecture

06

ViewModel

Menyimpan dan mengelola UI state

CAPAIAN PEMBELAJARAN

CPMK0501

Mahasiswa mampu menerapkan konsep pemrograman untuk pengembangan perangkat lunak.

Indikator: Mampu mengelola state dan menerapkan MVVM pattern

Setelah pertemuan ini, mahasiswa mampu:

- ✓ Memahami konsep state dalam Compose dan cara mengelolanya
- ✓ Menggunakan remember dan mutableStateOf dengan benar
- ✓ Menerapkan state hoisting untuk komponen yang reusable
- ✓ Memahami kapan dan mengapa recomposition terjadi
- ✓ Mengimplementasikan MVVM pattern dengan ViewModel

REVIEW MINGGU LALU

Pertemuan 3: Compose Multiplatform Basics

@Composable

Fungsi yang mendeskripsikan UI, menggunakan PascalCase

Layouts

Column (vertikal), Row (horizontal), Box (stack)

Modifiers

Chain untuk styling, urutan penting!

UI Components

Text, Button, Image, Card, TextField

? Pertanyaan: Bagaimana caranya agar UI berubah ketika data berubah?

BAGIAN 1

STATE DALAM COMPOSE

Data yang dapat berubah dan mempengaruhi UI

APA ITU STATE?

Definisi State

State adalah data yang dapat berubah seiring waktu dan mempengaruhi apa yang ditampilkan di UI. Ketika state berubah, Compose akan otomatis me-render ulang bagian UI yang terpengaruh.

Contoh State dalam Aplikasi:

Counter app	→ Nilai counter (0, 1, 2, ...)
Login form	→ Teks di TextField (username, password)
Todo list	→ List item dan status checked
Dark mode	→ Boolean (isDarkMode = true/false)
Loading indicator	→ Boolean (isLoading = true/false)

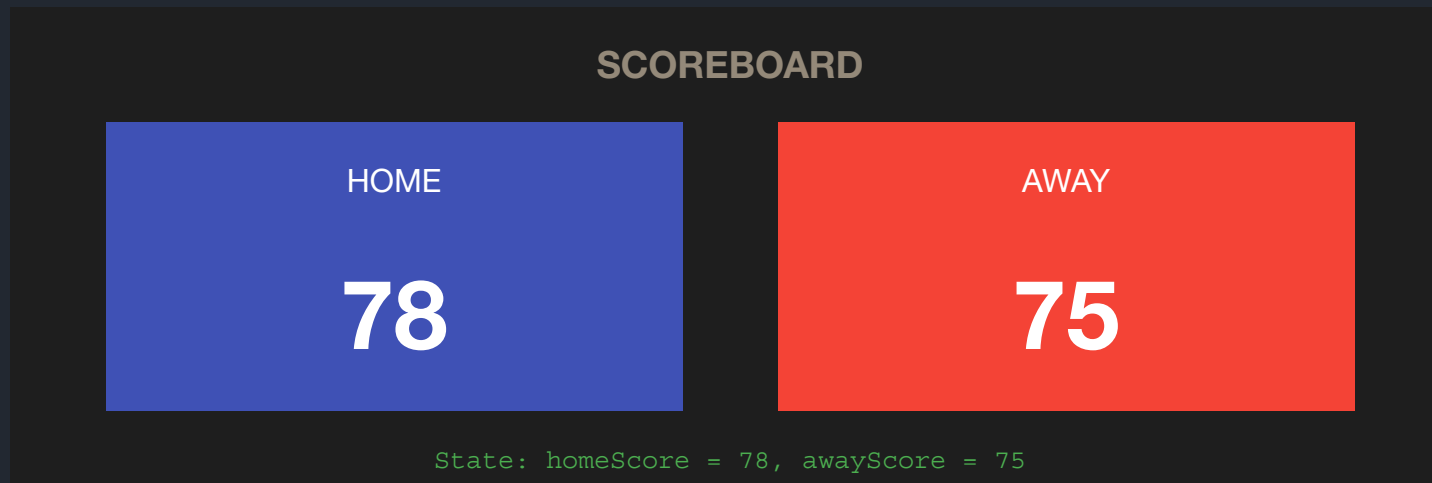
ANALOGI: PAPAN SKOR PERTANDINGAN

Bayangkan Papan Skor di Pertandingan Basket

- Papan skor = UI (tampilan)
- Skor tim = State (data yang berubah)
- Ketika ada gol, skor berubah → papan skor otomatis update
- Penonton tidak perlu "refresh" papan skor secara manual

Dalam Compose:

- Composable = Papan skor
- State = Skor
- Perubahan state → UI otomatis di-update (recomposition)



MASALAH: UI TANPA STATE

✗ Kode yang SALAH

```
@Composable
fun Counter() {
    var count = 0 // ✗ Variable biasa, TIDAK akan trigger recomposition

    Column {
        Text("Count: $count")

        Button(onClick = {
            count++ // ✗ Nilai berubah, tapi UI TIDAK update!
            println("Count is now $count") // Di console berubah
        }) {
            Text("Increment")
        }
    }
}

// MASALAH:
// 1. Variable 'count' akan di-reset setiap recomposition
// 2. Perubahan 'count' tidak trigger recomposition
// 3. UI tetap menampilkan "Count: 0" selamanya
```

⚠ Variable biasa akan di-reset setiap recomposition dan perubahan tidak trigger UI update!

SOLUSI: remember DAN mutableStateOf

✓ Kode yang BENAR

```
@Composable
fun Counter() {
    // ✓ remember: Menyimpan nilai agar survive recomposition
    // ✓ mutableStateOf: Membuat state yang observable
    var count by remember { mutableStateOf(0) }

    Column {
        Text("Count: $count")

        Button(onClick = {
            count++ // ✓ Perubahan akan trigger recomposition!
        }) {
            Text("Increment")
        }
    }
}

// PENJELASAN:
// remember    → Nilai disimpan di memory, tidak di-reset saat recomposition
// mutableStateOf → Compose "mengawasi" nilai ini, jika berubah → recomposition
```

✓ Gunakan `remember { mutableStateOf(...) }` untuk state yang mempengaruhi UI!

3 CARA MENDEKLARASIKAN STATE

1. Dengan Delegation (by) - RECOMMENDED

```
var count by remember { mutableStateOf(0) }  
count++ // Langsung akses nilai
```

2. Dengan Destructuring

```
val (count, setCount) = remember { mutableStateOf(0) }  
setCount(count + 1) // Gunakan setter function
```

3. Dengan .value (Explicit)

```
val count = remember { mutableStateOf(0) }  
count.value++ // Akses lewat .value
```

💡 Cara 1 (by) paling sering digunakan karena sintaksnya paling bersih

TIPE-TIPE STATE YANG UMUM

Berbagai Tipe State

```
// Boolean state
var isLoading by remember { mutableStateOf(false) }
var isDarkMode by remember { mutableStateOf(true) }

// String state
var username by remember { mutableStateOf("") }
var searchQuery by remember { mutableStateOf("") }

// Number state
var count by remember { mutableStateOf(0) }
var price by remember { mutableStateOf(0.0) }

// List state (gunakan mutableStateListOf)
val todoItems = remember { mutableStateListOf<TodoItem>() }
todoItems.add(TodoItem("New task")) // Trigger recomposition

// Object state
var selectedUser by remember { mutableStateOf<User?>(null) }
```

BAGIAN 2

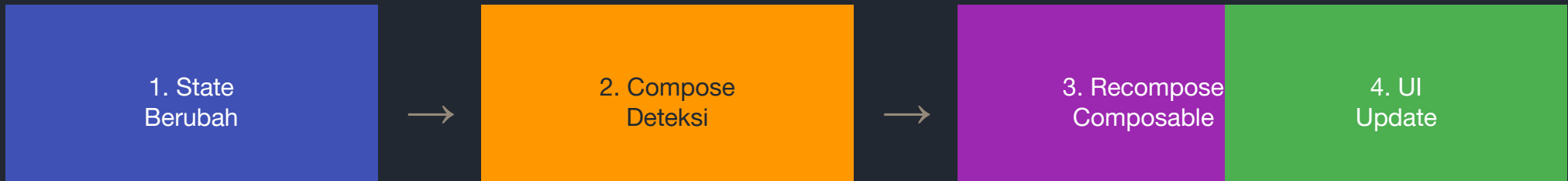
RECOMPOSITION

Bagaimana Compose mengupdate UI

APA ITU RECOMPOSITION?

Recomposition adalah proses di mana Compose memanggil ulang fungsi @Composable ketika state yang dibaca oleh fungsi tersebut berubah. Hanya composable yang terpengaruh yang di-recompose.

Alur Recomposition:



Poin Penting tentang Recomposition:

- Cepat dan efisien - hanya composable yang terpengaruh yang di-recompose
- Bisa terjadi berkali-kali - jangan lakukan side effect di dalamnya
- Smart recomposition - Compose skip composable yang inputnya tidak berubah

SCOPE RECOMPOSITION

Code

```
@Composable
fun ParentScreen() {
    var count by remember {
        mutableStateOf(0)
    }

    Column {
        // Recompose saat count berubah
        CounterDisplay(count = count)

        // Recompose saat count berubah
        IncrementButton(
            onClick = { count++ }
        )

        // TIDAK recompose! (tidak baca count)
        StaticText()
    }
}

@Composable
fun StaticText() {
    Text("I never change")
}
```

Visualisasi Recomposition:

ParentScreen

CounterDisplay ☹

IncrementButton ☹

StaticText ✓ Skip

☹ = Recompose, ✓ = Skip

BAGIAN 3

STATE HOISTING

Mengangkat state ke parent untuk reusability

APA ITU STATE HOISTING?

State Hoisting adalah pattern di mana state dipindahkan ke parent composable, sehingga child composable menjadi stateless dan lebih reusable. Child hanya menerima state dan callback untuk mengubah state.

Analogi: Remote TV

Remote TV tidak menyimpan channel saat ini (state). TV yang menyimpan channel-nya.
Remote hanya mengirim perintah "ganti ke channel X" (callback).
Remote bisa digunakan untuk TV manapun (reusable)!

Keuntungan State Hoisting:

✓ Reusable

Komponen bisa dipakai dengan state berbeda

✓ Testable

Mudah di-test karena tidak punya internal state

✓ Single Source of Truth

State hanya ada di satu tempat

STATEFUL vs STATELESS COMPOSABLE

Stateful (Internal State)

```
// ✗ State di dalam composable
@Composable
fun StatefulCounter() {
    var count by remember {
        mutableStateOf(0)
    }

    Column {
        Text("Count: $count")
        Button(
            onClick = { count++ }
        ) {
            Text("+1")
        }
    }
}

// Masalah:
// - Tidak reusable
// - Sulit di-test
// - Tidak bisa kontrol dari luar
```

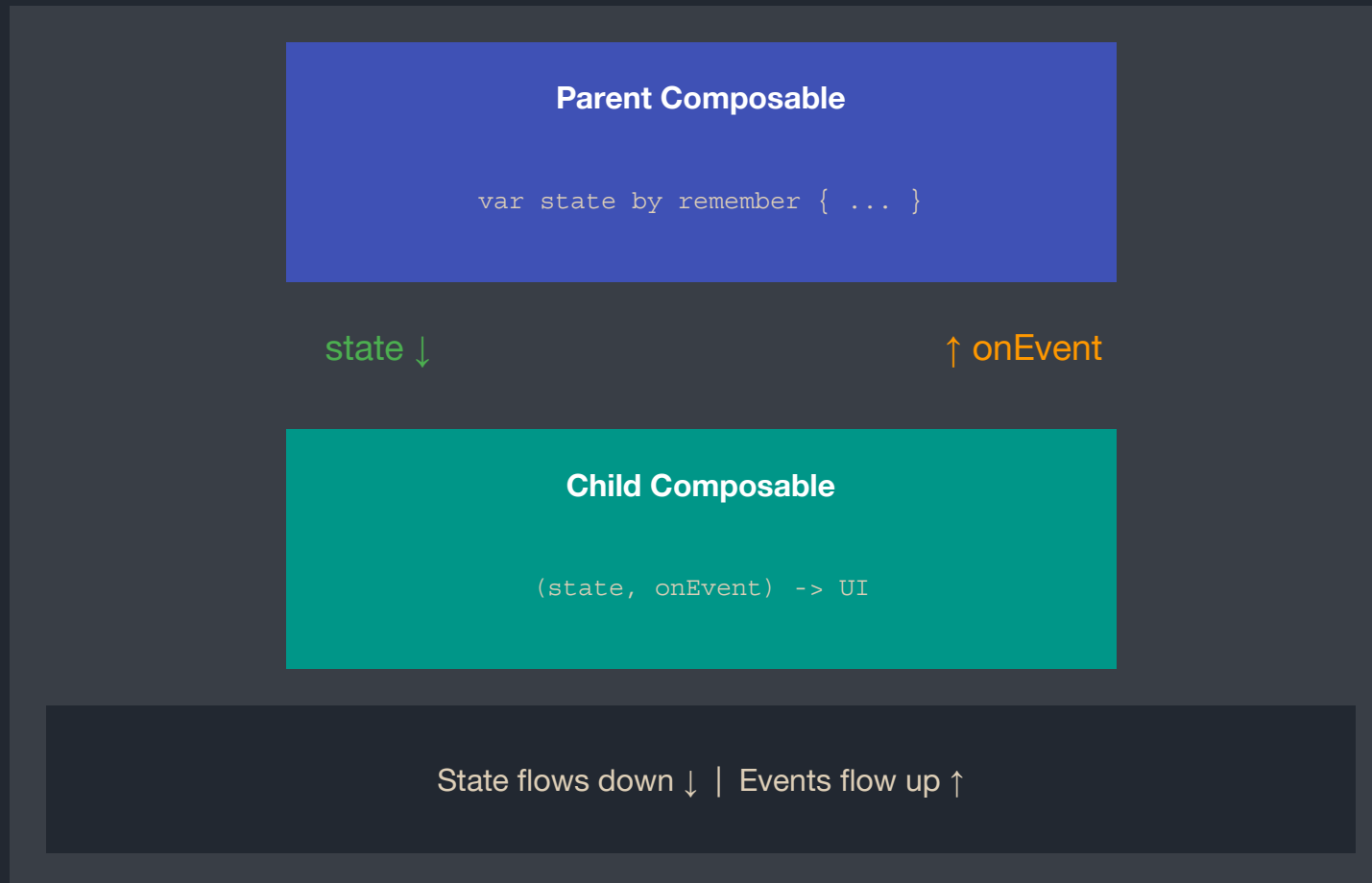
Stateless (Hoisted)

```
// ✓ State di-hoist ke parent
@Composable
fun StatelessCounter(
    count: Int,           // State
    onIncrement: () -> Unit // Callback
) {
    Column {
        Text("Count: $count")
        Button(onClick = onIncrement) {
            Text("+1")
        }
    }
}

// Penggunaan:
@Composable
fun ParentScreen() {
    var count by remember { mutableStateOf(0) }
    StatelessCounter(
        count = count,
        onIncrement = { count++ }
    )
}
```

PATTERN STATE HOISTING

Pola umum: state turun (↓), event naik (↑)



BAGIAN 4

MVVM PATTERN

Model - View - ViewModel

Architecture Pattern untuk Clean Code

APA ITU MVVM?

MVVM (Model-View-ViewModel) adalah architecture pattern yang memisahkan UI (View) dari business logic (ViewModel) dan data (Model). Pattern ini membuat kode lebih terstruktur, testable, dan maintainable.

MODEL

- Data classes
- Repository
- API/Database
- Business logic

VIEW

- @Composable
- UI Components
- User input
- Display state

VIEWMODEL

- UI State
- Event handlers
- Data transformation
- Survives config change

Data Flow: Model → ViewModel → View (UI)

Event Flow: View (User Action) → ViewModel → Model

ARSITEKTUR MVVM

VIEW (Composable)

- Menampilkan UI
- Menerima user input
- Observe state dari ViewModel

VIEWMODEL

- Menyimpan UI state
- Handle business logic
- Expose state ke View

MODEL (Repository)

- Data source (API, DB)
- Data classes

↓ State

↑ Events

↕ Data

Keuntungan MVVM:

- ✓ Separation of Concerns
UI terpisah dari logic
- ✓ Testability
ViewModel mudah di-unit test
- ✓ Maintainability
Perubahan di satu layer tidak affect layer lain
- ✓ Lifecycle Aware
ViewModel survives configuration changes

BAGIAN 5

VIEWMODEL IMPLEMENTATION

Implementasi ViewModel di KMP

APA ITU VIEWMODEL?

ViewModel adalah class yang menyimpan dan mengelola UI-related data. ViewModel survives configuration changes (seperti rotasi layar) sehingga data tidak hilang saat activity/fragment di-recreate.

✗ Tanpa ViewModel

- State hilang saat rotasi
- Data di-fetch ulang
- Loading state reset
- User input hilang
- Poor user experience

✓ Dengan ViewModel

- State preserved
- No duplicate API calls
- Loading state maintained
- User input preserved
- Smooth experience

Activity: onCreate → onDestroy → onCreate (rotation) → onDestroy

ViewModel: created —————> cleared

VIEWMODEL DI KOTLIN MULTIPLATFORM

Basic ViewModel (KMP)

```
// Di KMP, kita bisa menggunakan lifecycle-viewmodel-compose
// build.gradle.kts:
// implementation("org.jetbrains.androidx.lifecycle:lifecycle-viewmodel-compose:2.8.0")

class CounterViewModel : ViewModel() {
    // UI State menggunakan MutableStateFlow
    private val _count = MutableStateFlow(0)
    val count: StateFlow<Int> = _count.asStateFlow()

    // Event handlers
    fun increment() {
        _count.value++
    }

    fun decrement() {
        _count.value--
    }

    fun reset() {
        _count.value = 0
    }
}
```

UI STATE CLASS

Best Practice: Gunakan data class untuk UI State

UI State Pattern

```
// Data class untuk UI State
data class CounterUiState(
    val count: Int = 0,
    val isLoading: Boolean = false,
    val errorMessage: String? = null
)

class CounterViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(CounterUiState())
    val uiState: StateFlow<CounterUiState> = _uiState.asStateFlow()

    fun increment() {
        _uiState.update { currentState ->
            currentState.copy(count = currentState.count + 1)
        }
    }

    fun setLoading(loading: Boolean) {
        _uiState.update { it.copy(isLoading = loading) }
    }
}
```

MENGGUNAKAN VIEWMODEL DI COMPOSE

Compose + ViewModel

```
@Composable
fun CounterScreen(
    viewModel: CounterViewModel = viewModel() // Inject ViewModel
) {
    // Collect state as Compose State
    val uiState by viewModel.uiState.collectAsState()

    Column(
        modifier = Modifier.fillMaxSize().padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        if (uiState.isLoading) {
            CircularProgressIndicator()
        } else {
            Text(
                text = "Count: ${uiState.count}",
                fontSize = 32.sp
            )
        }

        Spacer(modifier = Modifier.height(16.dp))

        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            Button(onClick = { viewModel.decrement() }) { Text("-") }
            Button(onClick = { viewModel.increment() }) { Text("+") }
            Button(onClick = { viewModel.reset() }) { Text("Reset") }
        }
    }
}
```

CONTOH LENGKAP: TODO APP

Model

```
// Model
data class TodoItem(
    val id: Int,
    val title: String,
    val isCompleted: Boolean = false
)

// UI State
data class TodoUiState(
    val items: List<TodoItem> = emptyList(),
    val inputText: String = ""
)
```

ViewModel

```
class TodoViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(TodoUiState())
    val uiState = _uiState.asStateFlow()

    fun addTodo() { /* ... */ }
    fun toggleTodo(id: Int) { /* ... */ }
    fun deleteTodo(id: Int) { /* ... */ }
}
```

Preview UI:

My Todos

Add new todo...

☒ Buy groceries

☐ Learn Compose

☐ Build MVVM app

BAGIAN 6

HANDS-ON PRACTICE

3 Latihan Praktik

LATIHAN 1: COUNTER APP DENGAN STATE

Buat counter dengan increment, decrement, dan reset

Template

```
@Composable
fun CounterApp() {
    // TODO: Deklarasikan state untuk count
    // var count by remember { ... }

    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        // TODO: Tampilkan nilai count

        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            // TODO: Button decrement (-)
            // TODO: Button increment (+)
            // TODO: Button reset
        }
    }
}
```

Checklist:

- ☐ State dengan remember
- ☐ mutableStateOf(0)
- ☐ Text untuk display
- ☐ Button increment (+1)
- ☐ Button decrement (-1)
- ☐ Button reset (ke 0)
- ☐ Disable decrement jika 0

 Waktu: 15 menit

LATIHAN 2: FORM DENGAN STATE HOISTING

Buat reusable TextField component dengan state hoisting

Template

```
// Stateless TextField component
@Composable
fun LabeledTextField(
    label: String,
    value: String,           // State dari parent
    onChange: (String) -> Unit // Callback
) {
    // TODO: Implement OutlinedTextField
}

// Parent yang menyimpan state
@Composable
fun RegistrationForm() {
    var name by remember { mutableStateOf("") }
    var email by remember { mutableStateOf("") }

    Column {
        LabeledTextField(
            label = "Name",
            value = name,
            onChange = { name = it }
        )
        // TODO: Tambahkan email field
        // TODO: Tampilkan preview data
    }
}
```

Checklist:

- ☐ Stateless LabeledTextField
- ☐ Parameter: value, onChange
- ☐ State hoisted ke parent
- ☐ Name dan Email field
- ☐ Preview yang menampilkan "Hello, [name]!"
- ☐ Gunakan 1 komponen untuk kedua field

 Waktu: 20 menit

LATIHAN 3: TODO APP DENGAN VIEWMODEL

Implementasi MVVM pattern untuk Todo List sederhana

Template

```
// 1. Data class
data class Todo(val id: Int, val text: String, val done: Boolean)

data class TodoUiState(
    val todos: List<Todo> = emptyList(),
    val input: String = ""
)

// 2. ViewModel
class TodoViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(TodoUiState())
    val uiState = _uiState.asStateFlow()

    fun onInputChange(text: String) { /* TODO */ }
    fun addTodo() { /* TODO */ }
    fun toggleTodo(id: Int) { /* TODO */ }
}

// 3. Compose UI
@Composable
fun TodoScreen(viewModel: TodoViewModel = viewModel()) {
    // TODO: Implement UI
}
```

Checklist:

- ☐ Data class Todo
- ☐ TodoUiState dengan list
- ☐ TodoViewModel dengan StateFlow
- ☐ Add todo function
- ☐ Toggle done function
- ☐ TextField untuk input
- ☐ LazyColumn untuk list
- ☐ Checkbox untuk done

 Waktu: 30 menit

TROUBLESHOOTING

✗ UI tidak update saat state berubah

✓ Pastikan menggunakan `remember { mutableStateOf() }`

✗ State reset setiap recomposition

✓ Wrap dengan `remember { }`

✗ Unresolved reference: `by`

✓ `import androidx.compose.runtime.getValue/setValue`

✗ `collectAsState()` not found

✓ `import androidx.compose.runtime.collectAsState`

✗ ViewModel not found

✓ Add `lifecycle-viewmodel-compose` dependency

BEST PRACTICES

✓ State minimal

Simpan hanya data yang diperlukan UI, derive yang lain

✓ Single source of truth

State hanya ada di satu tempat (parent atau ViewModel)

✓ State hoisting

Hoist state ke level yang membutuhkan

✓ Immutable state

Gunakan data class dengan `copy()` untuk update

✓ Avoid side effects

Jangan lakukan network/DB call di composable langsung

✓ Use ViewModel

Untuk state yang survive configuration change

RANGKUMAN PERTEMUAN 4

State

Data yang berubah dan mempengaruhi UI

remember + mutableStateOf

Cara mendeklarasikan state di Compose

Recomposition

Proses update UI saat state berubah

State Hoisting

Pattern untuk membuat composable reusable

MVVM

Architecture pattern (Model-View-ViewModel)

ViewModel

Menyimpan state yang survive config change

🎯 **State + ViewModel = Reactive UI yang maintainable!**

TUGAS PRAKTIKUM MINGGU 4

Bobot: 4% | Deadline: Sebelum Pertemuan 5

Deskripsi Tugas:

Kembangkan Profile App dari minggu lalu dengan fitur:

1. Implementasi MVVM Pattern:
 - Buat ProfileViewModel dengan StateFlow
 - Data class ProfileUiState
2. Fitur Edit Profile:
 - Form untuk edit nama dan bio
 - State hoisting untuk TextField
 - Save button yang update ViewModel
3. Fitur Dark Mode Toggle:
 - Switch untuk dark/light mode
 - State disimpan di ViewModel

Format Pengumpulan:

- Push ke GitHub repository yang sama
- Screenshot: Profile view, Edit form, Dark mode
- Struktur folder: ui/, viewmodel/, data/

RUBRIK PENILAIAN TUGAS

Komponen	Bobot	Kriteria
ViewModel Implementation	25%	ProfileViewModel dengan StateFlow
UI State Pattern	20%	Data class ProfileUiState
State Hoisting	20%	Stateless TextField components
Edit Feature	20%	Form edit yang fungsional
Code Structure	15%	Folder structure, clean code
 Bonus (+10%)		
Dark mode theme yang smooth		
 Penalti		
Terlambat: -10%/hari, Plagiat: 0		

SUMBER PUSTAKA

Dokumentasi

State and Jetpack Compose

developer.android.com/jetpack/compose/state

Dokumentasi

ViewModel Overview

developer.android.com/topic/libraries/architecture/viewmodel

Guide

Guide to app architecture

developer.android.com/topic/architecture

Codelab

State in Jetpack Compose

developer.android.com/codelabs/jetpack-compose-state

KMP

Lifecycle ViewModel Compose

github.com/ArkiveFang/lifecycle-viewmodel-compose

PREVIEW MINGGU DEPAN

Pertemuan 5: Navigasi Antar Layar

Navigation Compose

Navigasi deklaratif di Compose

Routes dan Destinations

Mendefinisikan tujuan navigasi

Passing Arguments

Mengirim data antar screen

Back Stack

Mengelola history navigasi

Persiapan:

- Selesaikan tugas Profile App dengan MVVM
- Baca dokumentasi Navigation Compose
- Pastikan memahami state hoisting



SESI TANYA JAWAB

Ada pertanyaan seputar State, MVVM, atau ViewModel?

Terima Kasih

Pertemuan 4: State Management dan MVVM

"State management is the heart of any reactive UI framework."

- Android Developers

Selamat membangun aplikasi dengan arsitektur yang baik! 🏗️